FlexQL

A Flexible SQL-like Database Driver

Design Document

GitHub:Repository:
`https://github.com/skm5786/FlexQL-A_Flexible_SQL-like_Database_Driver`
Language: C++17
Build system: GNU Make
operating system: Linux
Name: Sumit
Roll no: 25CS60R63

Contents

1 System Overview **2**
1.1 High-Level Architecture 2

2 Wire Protocol and Data Flow **2**
2.1 Message Format 2
2.2 Request–Response Flow 3

3 Storage Design **3**
3.1 Storage Format: Row-Major 3
3.2 In-Memory Representation 4
3.3 Arena Allocator 4
3.4 Schema Storage 4

4 Indexing **4**
4.1 Primary Key Hash Index 4
4.2 B+ Tree (Range Queries) 5

5 Caching **5**
5.1 LRU Query Result Cache 5

6 Expiration Timestamps **5**

7 Multithreading Design **6**
7.1 Server Thread Model 6
7.2 Locking Hierarchy 6
7.3 Safe Concurrent Insert 6

8 Write-Ahead Log (WAL) **6**

9 Query Execution Pipeline **7**
9.1 Normal Path 7
9.2 Fast INSERT Path 7

10 Supported SQL Subset **8**

11 Client API **8**

12 Performance Decisions **9**

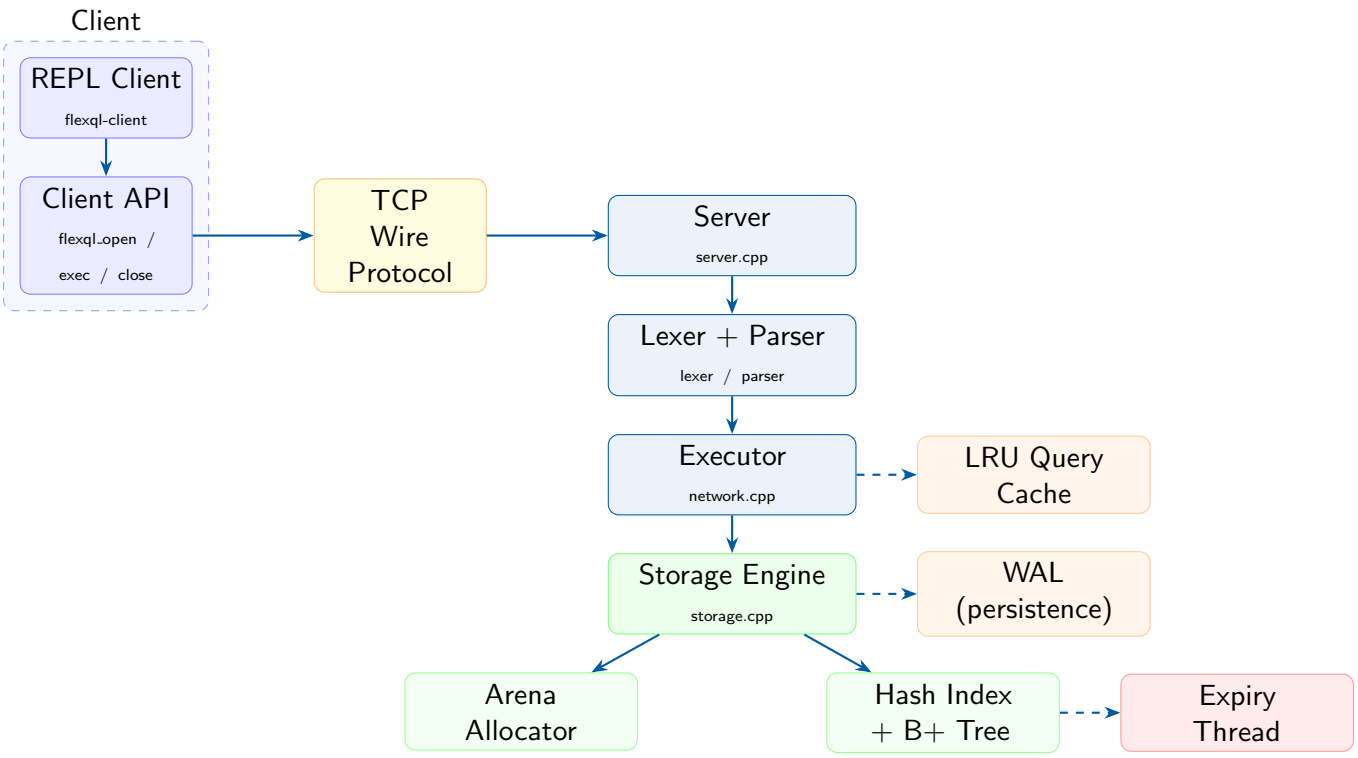
13 Compilation and Execution **9**

System Overview

FlexQL is a clientserver relational database engine implemented entirely in C++17. A TCP server process hosts all database state in memory and handles concurrent client connections, each on its own thread. Clients communicate with the server through a thin binary wire protocol, issuing SQL-like statements and receiving typed result rows.

The system supports a purposeful subset of SQL: `CREATE TABLE`, `DROP TABLE`, `INSERT INTO ... VALUES`, `SELECT` with an optional single-condition `WHERE` clause, and `INNER JOIN` between two tables.

High-Level Architecture



Wire Protocol and Data Flow

Message Format

Every message on the wire has a fixed 5-byte header followed by a variable-length payload.

```
[ 1 byte  ]  MessageType  (QUERY=0x01, RESULT=0x02, DONE=0x03,
                           ERROR=0x04, OK=0x05, ABORT=0x06)
[ 4 bytes ]  payload_len  (big-endian uint32)
[ N bytes ]  payload
```

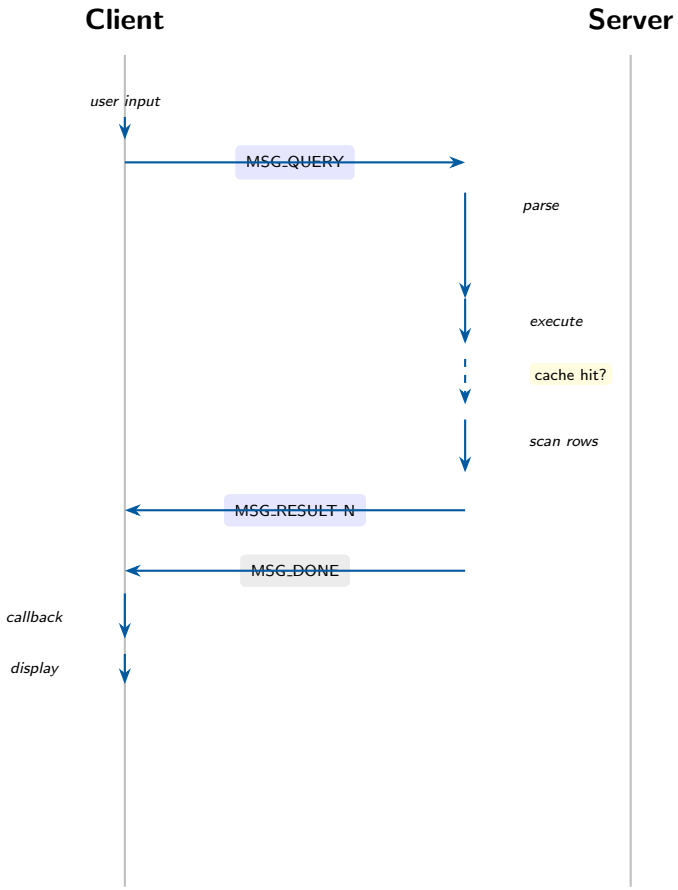
Result rows (MSG_RESULT payload) are serialised as:

```
[ 4 bytes ]  col_count
for each column:
```

```
[ 4 bytes ]  name_len
[ name_len ] column name (UTF-8)
[ 4 bytes ]  val_len     (0 = SQL NULL)
[ val_len  ] value string
```

Request–Response Flow

The diagram below shows the full lifecycle of a single SQL statement from the moment it is typed in the REPL until the result is displayed.



For a **non-SELECT** statement (INSERT, CREATE TABLE) the server sends a single `MSG_OK` instead of `MSG_RESULT` rows followed by `MSG_DONE`.

Storage Design

Storage Format: Row-Major

We chose **row-major** storage. Each row is represented as a `Row` struct holding a heap of `CellValue` unions (one per column). Rows are linked in an insertion-order singly-linked list hanging off each `Table`.

Why row-major? The dominant operations are full-row inserts and full-row lookups/scans. Row-major keeps all columns of one row together, so a single row can be marshalled to the client in one pass without gathering from multiple column arrays. Column-major

would benefit analytical aggregations (`SUM`, `AVG` on one column), which are outside our supported SQL subset.

In-Memory Representation

```
struct Table {
    ColumnDef        schema[64];   // type + constraint per column
    int              col_count, pk_col;
    Row             *head, *tail;  // insertion-order linked list
    uint64_t         row_count;
    pthread_rwlock_t lock;         // readers share; writers exclusive
    HashIndex       *pk_index;     // O(1) PK equality lookup
    BTree           *col_btree[64];// O(log n+k) range on INT/DECIMAL
    Arena           *row_arena;    // bump allocator, all row memory
};
```

Arena Allocator

All `Row` structs, `CellValue` arrays, and VARCHAR strings are allocated from a per-table arena rather than via individual `malloc` calls.

The arena is a singly-linked list of fixed-size slabs (16 MB each). Allocation is a single pointer-bump no lock, no free-list. All memory is released at once when the table is dropped via `arena_free()`.

This eliminates ∼2 000 slab-level `malloc` calls and $N \times$ `col_count` per-row allocations that would otherwise occur over a 10 M-row insert run.

Schema Storage

Each `Table` holds a fixed-size `ColumnDef schema[64]` array storing the name, type (`INT`, `DECIMAL`, `VARCHAR`, `DATETIME`), constraint flags (`NOT NULL`, `PRIMARY KEY`), and ordinal index for each column. Type enforcement happens in `row_insert()` via `string_to_cell()`, which converts raw string values from the parser into typed `CellValue` unions and rejects mismatches before anything is written to the linked list.

Indexing

Primary Key Hash Index

We use **open-addressing** with **linear probing** and **tombstone deletion**.

- Capacity is always a power of two; load factor is capped at 0.7.
- Integer keys use a Knuth multiplicative hash (`key` $\times$ 11400714819323198485), truncated with a bitmask.
- VARCHAR keys use djb2a.
- When load factor is exceeded, the slot array is doubled and all live entries are re-inserted (tombstones are discarded).

For bulk-insert workloads the initial capacity is pre-sized to $\lceil N/0.7 \rceil$ rounded up to the next power of two, eliminating all resize operations. For a 10 M-row table this avoids 23 resize-copy operations that would together move ∼2.3 GB of slot data.

B+ Tree (Range Queries)

One B+ tree is maintained per non-PK `INT` or `DECIMAL` column. It handles range predicates such as `WHERE balance > 1000` in $O(\log n + k)$ time instead of $O(n)$.

Key design decisions:

- **Order 32** each node holds 3163 keys; tree height for 10 M rows is $\lceil \log_{32}(10^7) \rceil \approx 4$ levels.
- **Keys stored as `double`** `int64_t` values up to 2^{53} are representable exactly; DECIMAL values are stored natively.
- **Leaf doubly-linked list** leaves are threaded left-to-right so a range scan walks k results sequentially without backtracking.
- **Lazy population** above 50 000 rows, B+ tree inserts are deferred. The tree is rebuilt from the linked list on the first range query that needs it. This removes three `btree_insert()` calls per row from the bulk-insert hot path.

Caching

LRU Query Result Cache

FlexQL caches the serialised wire payloads of SELECT results in an **LRU (Least Recently Used)** cache.

Structure: doubly-linked list (access order) + hash map (SQL key → node pointer), giving $O(1)$ hit, miss, and eviction.

Key: `"DBNAME:SQL"` (upper-cased so SQL is case-insensitive).

Value: a singly-linked list of serialised `MSG_RESULT` byte buffers the exact bytes that will be sent to the client. On a cache hit, these are re-sent verbatim with zero storage access.

Capacity: 256 entries or 64 MB of payload bytes, whichever is reached first.

Invalidation: Any INSERT into table T evicts all cache entries whose key contains the name of T. DROP TABLE and DROP DATABASE evict the entire table/database namespace.

Why LRU over LFU or FIFO? LRU is $O(1)$, simple to implement correctly, and performs well for query workloads where recently executed queries tend to repeat (e.g. dashboards, repeated lookups).

Expiration Timestamps

Every inserted row carries an `expiry` field (Unix timestamp, 0 = never). Expired rows are **soft-deleted**: they remain in the linked list but are skipped during `table_scan()` when their `expiry < time(nullptr)`.

A dedicated **background expiry thread** wakes every 30 seconds, acquires a write lock on each table, and physically unlinks expired rows from the list, removing them from the hash index at the same time. This reclaims their slot in the linked list (though their arena memory is only reclaimed when the table is dropped, since the arena does not support per-object free).

Soft deletion means a PK value from an expired row can be re-inserted: the duplicate check in `row_insert()` treats an expired existing row as absent and removes its index entry before proceeding.

Multithreading Design

Server Thread Model

The server uses a **one-thread-per-connection** model. The main thread runs a `select()`-based accept loop and spawns a detached `pthread` for each incoming connection. Each thread owns its own `Database` session and communicates with the shared `DatabaseManager` only through mutex-protected calls.

Locking Hierarchy

Lock	Type	Protects
`DatabaseManager.lock`	`pthread_mutex_t`	The global database registry (`databases[]` array). Held only for CREATE/DROP/FIND database.
`Database.schema_lock`	`pthread_mutex_t`	The table array within a database. Held for CREATE/DROP table and table lookup.
`Table.lock`	`pthread_rwlock_t`	Row linked list, hash index, row count. Read-locked during SELECT/scan; write-locked during INSERT and expiry purge.
`LRUCache.lock`	`pthread_mutex_t`	All cache operations (get, put, invalidate).
`g_wal_mutex`	`pthread_mutex_t`	The WAL write-buffer table (all connections share one buffer pool).

The hierarchy is always acquired top-down (Manager → Database → Table), preventing deadlock by design.

Safe Concurrent Insert

For INSERT the write lock on `Table` is held for the minimal critical section: list append and hash index update. The PK duplicate check first uses a **read lock** (allowing concurrent readers to proceed) and only escalates to a write lock if an expired entry must be removed. Row ID assignment is done with `__sync_fetch_and_add` outside the lock. B+ tree inserts for small tables happen outside the write lock entirely (the row pointer is stable once written to the arena).

Write-Ahead Log (WAL)

FlexQL writes an append-only WAL for crash recovery.

Location: `data/<db_name>/<table_name>.wal` per table.

Record format:

```
[8 bytes]  LSN (log sequence number, monotonically increasing)
[4 bytes]  entry type  (CREATE_TABLE=1, INSERT=2, DROP_TABLE=3)
[4 bytes]  payload length
[N bytes]  payload (binary-encoded row data)
[4 bytes]  CRC32 checksum
```

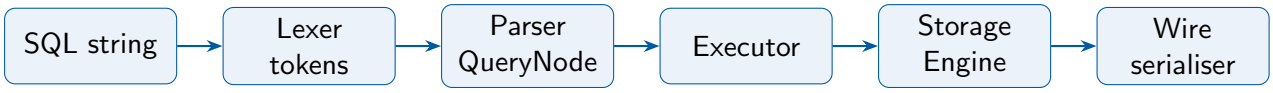
Buffered group commit: WAL records for INSERT are accumulated in a per-table write buffer and flushed to disk (with `fdatasync`) when either 5 000 rows have been buffered or 200 ms have elapsed. This amortises `fdatasync` cost across many rows.

Recovery: At startup, `wal_recover()` reads `data/_registry` to discover named databases, then replays each `.wal` file in order, reconstructing the in-memory table and row state. CRC32 mismatches (due to a truncated write during a crash) halt replay for that file without corrupting the rest.

Per-connection sessions: Each client connection gets its own `BENCH_<fd>` database. On a clean disconnect the WAL files and the session database are deleted. If the server crashes, the WAL files survive and `wal_recover()` replays them on the next start.

Query Execution Pipeline

Normal Path



Lexer single-pass scanner; SQL keywords are case-folded to uppercase; single-quoted strings are unescaped; numbers are returned as text tokens.

Parser recursive descent; produces a `QueryNode` union (`CreateTableParams`, `InsertParams`, `SelectParams`, etc.). `InsertParams` supports batch VALUES tuples in one statement.

Executor (`network.cpp`) dispatches on `QueryNode.type`, checks the LRU cache for SELECT, writes WAL for INSERT/CREATE, calls the storage engine, then serialises and sends result rows.

Fast INSERT Path

For the common `INSERT INTO t VALUES (...)` pattern a hand-written scanner (`fast_insert.cpp`) is tried **before** the full lexer/parser.

It converts numeric literals to `int64_t`/`double` exactly once and writes typed `CellValues` directly into the arena, bypassing:

- Full lexer tokenisation (∼6 000 tokens for a 500-row batch).
- `InsertParams` heap allocation (each extra row is `char[64][4096]`).
- `string_to_cell()` → `strtod`/`strtoll` calls in the storage layer.

If the statement is not in the recognised pattern (column lists, expressions), the scanner returns `FAST_INSERT_FALLBACK` and the normal path is used.

Supported SQL Subset

Statement	Notes
`CREATE TABLE t (...)`	Columns typed as `INT`, `DECIMAL`, `VARCHAR`(n), `DATETIME`. Constraints: `NOT NULL`, `PRIMARY KEY`.
`DROP TABLE t`	Frees arena + indexes immediately.
`INSERT INTO t VALUES (...)`	Batch form (`VALUES (...), (...), ...`) supported. Each row may carry an optional expiry timestamp.
`SELECT * / cols FROM t`	Optional single-condition `WHERE col op val`. Operators: `=`, `!=`, `<`, `>`, `<=`, `>=`. No AND/OR.
`SELECT ... INNER JOIN`	Equi-join on one column pair. Optional `WHERE` on the join result.
`CREATE / DROP DATABASE`	Multi-database support via `USE db`.
`SHOW DATABASES / TABLES`	Lists available databases or tables.

Client API

The public library API uses the opaque-pointer pattern identical to SQLite3:

Function	Description
`flexql_open(host, port, &db)`	Allocates a `FlexQL*` handle, opens a TCP connection.
`flexql_exec(db, sql, cb, arg, &err)`	Sends SQL; invokes `cb(arg, colCount, values, colNames)` once per result row; returns `FLEXQL_OK` or `FLEXQL_ERROR`.
`flexql_close(db)`	Closes the socket; frees the handle.
`flexql_free(ptr)`	Frees library-allocated memory (e.g. error strings).

Error codes: `FLEXQL_OK`=0, `FLEXQL_ERROR`=1, `FLEXQL_NOMEM`=2, `FLEXQL_TIMEOUT`=3, `FLEXQL_ABORT`=5.

Performance settings applied to the client socket: `TCP_NODELAY` to eliminate Nagle buffering, and 256 KB send/receive buffers to absorb large batch INSERT payloads without stalling.

Performance Decisions

Decision	Rationale
16 MB arena slabs	At 1 MB, inserting 10 M rows causes ∼2 000 slab allocations. At 16 MB: only ∼125 fewer syscalls and better cache locality since rows within a slab are physically contiguous.
Pre-sized hash index	Starting from capacity 256 causes 23 resize operations for 10 M rows, each copying hundreds of MB. Pre-sizing to 16 M slots eliminates all resizes at the cost of one larger upfront allocation.
Lazy B+ tree	Three `btree_insert()` calls per row at 10 M rows add ∼6 s if the workload never issues range queries (as in a pure insert benchmark). Trees are rebuilt on first use.
Batch VALUES + `TCP_NODELAY`	Sending 500 rows per TCP round-trip instead of 1 reduces round-trips from 10 M to 20 000 for a 10 M-row insert. `TCP_NODELAY` on both client and server eliminates the Nagle/delayed-ACK interaction that added ∼40 ms latency per small send.
WAL group commit	Flushing every 5 000 rows instead of every row reduces `fdatasync` calls from 10 M to 2 000 each call costs 14 ms on a spinning disk.
Compiler flags	`-O3 -march=native` enables auto-vectorisation and CPU-specific instruction selection for `memcpy`, `strcmp`, and `strtod` the dominant inner-loop operations during INSERT.

Compilation and Execution

```
# Build everything
make

# Start the server (default port 9000)
./bin/flexql-server 9000

# Connect with the interactive REPL
./bin/flexql-client 127.0.0.1 9000

# Debug build (AddressSanitizer + UBSan)
make debug
```